Математика алгоритмов

Конспект-практикум для программистов

Цель: понять, какие бывают алгоритмы, как они работают, как сравнивать их по скорости и где они применяются.

Как работать с конспектом

- Синие блоки «**Задание**» — обязательные.
  Уровень: ●○○ лёгкое, ●●○ среднее, ●●● сложное.
- Оранжевые блоки «**Подумайте сами**» — сначала ответьте, потом читайте дальше.
- Ответы к заданиям собраны в конце. Сначала решите сами!

1. Что такое алгоритм

Определение

Алгоритм — точная пошаговая инструкция, которая за конечное число шагов превращает входные данные в результат.

Хороший алгоритм:

- **понятен**: на каждом шаге ясно, что делать;
- **конечен**: когда-нибудь останавливается;
- **универсален**: работает для любых подходящих данных, а не для одного примера.

Про любой алгоритм мы задаём три вопроса:

Вопрос	Что это значит
Верно ли?	всегда ли он даёт правильный ответ
Быстро ли?	как растёт время работы, когда данных становится больше
Можно ли лучше?	есть ли способ быстрее или проще

Интересный факт

Слово «алгоритм» пришло от имени учёного аль-Хорезми (IX век). А алгоритм Евклида, который мы разберём ниже, придуман больше 2000 лет назад и работает до сих пор.

2. Как измерять скорость алгоритма

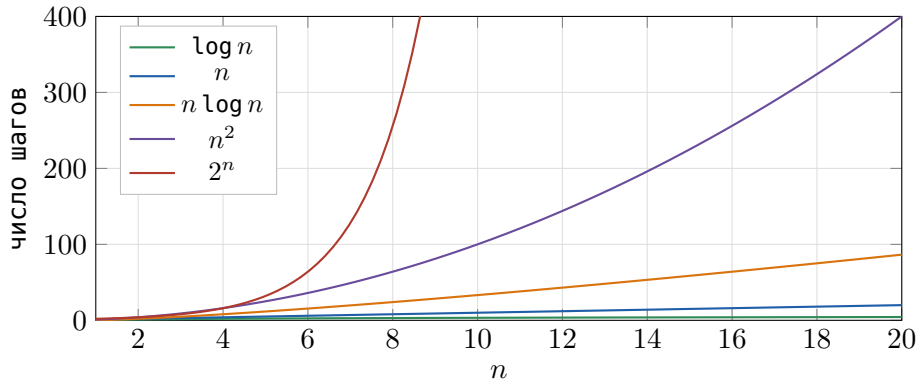
Секунды зависят от компьютера. Поэтому мы считаем **шаги** и смотрим, как их число растёт вместе с размером данных n. Это называют *порядком роста* и записывают как $\mathcal{O}(\dots)$ (читается «о большое»).

Простые правила подсчёта:

- один цикл по n элементам — $\mathcal{O}(n)$ шагов;
- цикл внутри цикла — шаги *умножаются*: $n \cdot n = \mathcal{O}(n^2)$;
- если счётчик каждый раз удваивается (i *= 2) — около $\log_2 n$ шагов (сколько раз можно разделить n на 2);
- константы и мелкие слагаемые отбрасываем: $3n + 10$ это просто $\mathcal{O}(n)$.

Сколько это на практике? Пусть компьютер делает 10^8 простых операций в секунду.

Порядок роста	Пример	Время для $n = 10^6$
$\mathcal{O}(\log n)$	бинарный поиск	20 шагов, мгновенно
$\mathcal{O}(n)$	один проход по списку	0.01 с
$\mathcal{O}(n \log n)$	хорошая сортировка	0.2 с
$\mathcal{O}(n^2)$	пузырёк	**около 3 часов**
$\mathcal{O}(2^n)$	перебор всех подмножеств	при $n = 50$: **130 суток**



Подумайте сами

Данных стало в 2 раза больше. Во сколько раз вырастет время у алгоритма $\mathcal{O}(n)$? А у $\mathcal{O}(n^2)$?

Задание 1 ●○○

Сколько раз выполнится s += 1 в каждом фрагменте? Запишите порядок роста.

```python
# (a)
for i in range(n):
    s += 1
# (b)
for i in range(n):
    for j in range(n):
        s += 1
# (c)
i = 1
while i < n:
    s += 1
    i *= 2
```

3. Поиск

Линейный поиск: смотрим элементы подряд, пока не найдём. В худшем случае проверим все n элементов — $\mathcal{O}(n)$. Работает на любом списке.

Бинарный поиск работает, как игра «угадай число от 1 до 100»: вы называете *середину*, вам говорят «больше» или «меньше», и половина вариантов сразу отпадает. Условие: список должен быть **отсортирован**.

```python
def bsearch(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1       # ищем правее
        else:
            hi = mid - 1       # ищем левее
    return -1                  # не нашли
```

```cpp
int binary_search_idx(const vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;      // безопаснее, чем (lo + hi) / 2
        if (a[mid] == x) return mid;
        if (a[mid] < x)  lo = mid + 1;
        else             hi = mid - 1;
    }
    return -1;
}
```

Осторожно

В C++ сумма lo + hi может не поместиться в int, поэтому середину считают как lo + (hi - lo) / 2.

Пример. Список $[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$, ищем 23:

шаг	lo	hi	mid	a[mid]	решение
1	0	9	4	16	$16 < 23$: идём вправо
2	5	9	7	56	$56 > 23$: идём влево
3	5	6	5	23	найдено!

Почему это так быстро? Каждый шаг отбрасывает половину. Значит, шагов не больше, чем раз можно поделить n на 2:

размер списка	1000	10^6	10^9
шагов не больше	10	20	30

В списке из миллиарда элементов нужное число найдётся за 30 шагов!

Где применяется

git bisect — ищет коммит, который сломал программу, бинарным поиском по истории. Поиск в базах данных (индексы), функции bisect в Python и lower_bound в C++.

Подумайте сами

Почему бинарный поиск не работает на неотсортированном списке? Что именно мы «выбрасываем» после сравнения с серединой?

Задание 2 ●●○

Реализуйте бинарный поиск и добавьте счётчик шагов. Проверьте на списке из примера: $x = 23$ и $x = 7$.

Задание 3 ●○○

Сколько шагов *максимум* понадобится бинарному поиску для списка из 1000 элементов? Из 10^6? Подсказка: сколько раз можно разделить число на 2, пока не останется 1?

4. Сортировки

4.1 Пузырёк

Идём по списку и меняем местами соседей, если они стоят неправильно. После первого прохода самый большой элемент “всплывает” в конец, после второго — второй по величине, и так далее.

```
def bubble(a):
    n = len(a)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
```

Сравнений: $(n-1)+(n-2)+\cdots+1$, то есть примерно $n^2/2$. Для 8 элементов их 28. Порядок роста — $\mathcal{O}(n^2)$.

4.2 Сортировка слиянием

Идея «разделяй и властвуй»:

1. делим список пополам;
2. сортируем каждую половину (так же, рекурсивно);
3. **сливаем** две отсортированные половины в одну: смотрим на первые элементы и берём меньший.

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    left, right = merge_sort(a[:m]), merge_sort(a[m:])
    res, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            res.append(left[i]); i += 1
```

```
        else:
            res.append(right[j]); j += 1
    return res + left[i:] + right[j:]
```

Пример для $[5, 2, 4, 7, 1, 3, 2, 6]$:

делим: $[5, 2, 4, 7 \mid 1, 3, 2, 6] \to [5, 2 \mid 4, 7]\,[1, 3 \mid 2, 6] \to$ по одному элементу
сливаем: $[2, 5]\,[4, 7]\,[1, 3]\,[2, 6] \to [2, 4, 5, 7]\,[1, 2, 3, 6] \to [1, 2, 2, 3, 4, 5, 6, 7]$

Почему быстрее? Делить пополам можно только $\log_2 n$ раз (это количество «этажей»), а на каждом этаже слияния в сумме обходят все n элементов один раз. Итого $n \cdot \log_2 n$ шагов — $\mathcal{O}(n \log n)$.

Осторожно

Для $n = 10^6$ пузырёк делает около 10^{12} шагов, а слияние — около $2 \cdot 10^7$. Это разница между тремя часами и долей секунды.

Где применяется

Встроенная sorted() в Python использует Timsort (слияния + вставки), std::sort в C++ — тоже быстрый гибридный алгоритм $\mathcal{O}(n \log n)$. Сортировка нужна везде: поиск, рейтинги, удаление дубликатов, группировка.

Задание 4 ●●○

Замерьте время bubble и merge_sort на случайных списках длины 1000, 2000 и 4000 (модуль time). Во сколько раз растёт время, когда n удваивается? Совпало ли с ожиданием?

5. Жадные алгоритмы

Идея

Жадный алгоритм на каждом шаге берёт то, что выглядит лучшим *прямо сейчас*, и никогда не передумывает. Он простой и быстрый, но иногда ошибается.

5.1 Размен монет

Выдать сумму наименьшим числом монет: берём самую большую монету, которая помещается.

```
def greedy_coins(coins, s):
    cnt = 0
    for c in sorted(coins, reverse=True):
        cnt += s // c
        s %= c
    return cnt
```

Монеты $\{1, 5, 10, 25\}$, сумма 63: $25 + 25 + 10 + 1 + 1 + 1$ — 6 монет, и лучше нельзя. Но...

Осторожно

Монеты $\{1, 3, 4\}$, сумма 6. Жадный: $4+1+1$ — **3 монеты**. Лучше: $3+3$ — **2 монеты**. Жадность ошиблась!

Вывод: жадный алгоритм нужно проверять на примерах, особенно на «подлых».

5.2 Выбор заявок

Есть мероприятия с временем начала и конца. Нужно выбрать как можно больше таких, которые не пересекаются. **Жадное правило:** сортируем по времени *окончания* и берём каждое, которое начинается не раньше конца последнего выбранного. (Идея: заканчиваем как можно раньше — остаётся больше места.)

```
def max_events(events):
    events.sort(key=lambda p: p[1])      # по времени окончания
    chosen, end = [], -1
    for s, f in events:
        if s >= end:
            chosen.append((s, f))
            end = f
    return chosen
```

Пример: $(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11),(8,12),(2,14),(12,16)$. Берём $(1,4)$, затем $(5,7)$, затем $(8,11)$, затем $(12,16)$. Итого **4 мероприятия**.

Где применяется

Кодирование Хаффмана (сжатие в ZIP и JPEG), составление расписаний, минимальное остовное дерево (прокладка сети кабелей с наименьшей длиной).

Задание 5 ●●○

Реализуйте max_events и проверьте на примере. Затем попробуйте другое жадное правило: «брать самое *короткое* мероприятие». Найдите набор заявок, на котором оно ошибается.

6. Динамическое программирование

Идея: если одна и та же мелкая задача решается много раз, давайте решим её *один раз* и запомним ответ.

6.1 Пример: числа Фибоначчи

Каждое число — сумма двух предыдущих: $0, 1, 1, 2, 3, 5, 8, 13, \ldots$

```
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Выглядит красиво, но работает ужасно: fib(5) вызывает функцию 15 раз, fib(25) — 242 785 раз, fib(40) — более 300 миллионов! Одни и те же значения считаются снова и снова.

Решение: запоминаем

```
from functools import lru_cache

@lru_cache(maxsize=None)      # запоминает уже посчитанные ответы
def fib_memo(n):
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

def fib_table(n):             # то же самое без рекурсии
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
```

```
    return a
```

Теперь fib(25) считается 26 раз вместо 242 785, а время — $\mathcal{O}(n)$.

6.2 Размен монет: динамика исправляет жадность

Пусть $dp[x]$ — наименьшее число монет для суммы x. Последняя монета — любая из набора, поэтому

$$dp[x] = 1 + \text{наименьшее из } dp[x-c] \text{ по всем монетам } c \leqslant x.$$

```
def dp_coins(coins, S):
    INF = 10**9
    dp = [0] + [INF] * S
    for x in range(1, S + 1):
        for c in coins:
            if c <= x:
                dp[x] = min(dp[x], dp[x - c] + 1)
    return dp
```

Таблица для монет $\{1,3,4\}$:

x	0	1	2	3	4	5	6
$dp[x]$	0	1	2	1	1	2	2

Например, $dp[6] = 1 + \min(dp[5], dp[3], dp[2]) = 1 + \min(2,1,2) = 2$ (монеты $3+3$). Динамика нашла ответ, на котором жадность ошиблась.

Подумайте сами

Заполните $dp[x]$ для $x = 7, 8, 9, 10$. Сколько монет нужно на 10 копеек? А сколько дал бы жадный алгоритм?

Где применяется

Сравнение текстов (diff, git), исправление опечаток (расстояние Левенштейна), сравнение ДНК, задача о рюкзаке (что взять с собой, чтобы получить максимум пользы).

Задание 6 ●●○

Лесенка. Подниматься можно на 1 или на 2 ступеньки за шаг. Сколькими способами можно подняться на n ступенек? Подумайте, как ответ для n связан с ответами для $n-1$ и $n-2$, и реализуйте динамикой. Какая знакомая последовательность получилась?

Задание 7 ●●○

Используя dp_coins и greedy_coins, найдите все суммы от 1 до 20, где для монет $\{1,3,4\}$ жадность даёт неоптимальный ответ.

7. Алгоритмы с числами

7.1 Алгоритм Евклида: наибольший общий делитель

Правило: НОД(a,b) = НОД$(b,\ a \bmod b)$, где $a \bmod b$ — остаток от деления. Повторяем, пока остаток не станет нулём. Почему работает: любое число, на которое делятся a и b, делит и остаток.

Пример: НОД$(1071, 462)$

$$1071 = 2 \cdot 462 + 147, \qquad 462 = 3 \cdot 147 + 21, \qquad 147 = 7 \cdot 21 + 0.$$

Остаток стал нулём, значит ответ — последний делитель: **21**.

```
long long gcd_ll(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Алгоритм очень быстрый: число шагов растёт как логарифм — для чисел порядка миллиарда хватает нескольких десятков шагов.

7.2 Быстрое возведение в степень

Чтобы вычислить a^{13}, не нужно делать 12 умножений. Заметим, что $13 = 8 + 4 + 1$, поэтому $a^{13} = a^8 \cdot a^4 \cdot a^1$. А a^2, a^4, a^8 получаются возведением в квадрат: $a \to a^2 \to a^4 \to a^8$. Всего несколько операций.

Для больших степеней берут остаток (mod), чтобы числа не росли. Пример: $3^{13} \bmod 1000$.

```
long long power_mod(long long a, long long n, long long m) {
    long long result = 1;
    a %= m;
    while (n > 0) {
        if (n & 1) result = result * a % m;   // этот разряд«» нужен
        a = a * a % m;                         // возводим в квадрат
        n >>= 1;                               // переходим к следующему разряду
    }
    return result;
}
```

n	нужен ли разряд	result	a
13	да	3	9
6	нет	3	81
3	да	$3 \cdot 81 = 243$	561
1	да	$243 \cdot 561 \to 323$	721

Ответ: $3^{13} \bmod 1000 = 323$. Шагов столько, сколько цифр в двоичной записи n — то есть $\mathcal{O}(\log n)$.

7.3 Решето Эратосфена: все простые числа до n

1. Выписываем числа от 2 до n.
2. Берём первое невычеркнутое число p — оно простое.
3. Вычёркиваем все его кратные, начиная с p^2.
4. Повторяем, пока $p \cdot p \leqslant n$.

Пример для $n = 30$. Вычёркиваем кратные 2, затем 3, затем 5 (дальше $7 \cdot 7 > 30$, стоп). Остались **2, 3, 5, 7, 11, 13, 17, 19, 23, 29**.

```
vector<int> sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (long long i = 2; i * i <= n; ++i)
        if (is_prime[i])
            for (long long j = i * i; j <= n; j += i)
                is_prime[j] = false;
    vector<int> primes;
    for (int i = 2; i <= n; ++i)
        if (is_prime[i]) primes.push_back(i);
    return primes;
}
```

Решето работает почти за $\mathcal{O}(n)$ — для миллиона чисел это доли секунды.

Где применяется

Шифрование в интернете (RSA в HTTPS) использует большие простые числа, НОД и возведение в степень по модулю. Хеширование, генераторы случайных чисел, сокращение дробей.

Задание 8 ●○○

Вручную выполните алгоритм Евклида для чисел 252 и 198. Сколько делений понадобилось?

Задание 9 ●●○

Реализуйте power_mod (на Python или C++) и проверьте: $2^{10} \bmod 1000$ и $3^{13} \bmod 1000$. Сравните с встроенным pow(a, n, m) в Python.

Задание 10 ●●○

Реализуйте решето. Сколько простых чисел не больше 100, 1000, 10^6?

8. Графы: кратчайший путь

Граф — это точки (вершины) и линии между ними (рёбра). Карта дорог — это граф: города и дороги.

8.1 Поиск в ширину (BFS)

Идея: обходим вершины «волнами», как круги на воде: сначала старт, потом его соседи, потом соседи соседей. Так находим путь с наименьшим *числом рёбер*. Используем очередь.

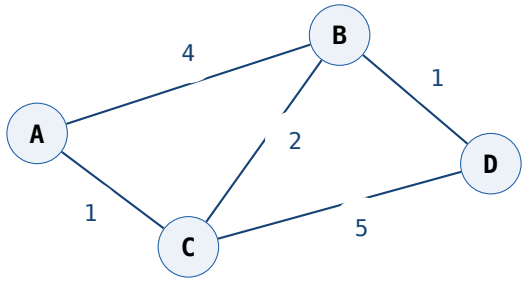
```
from collections import deque

def bfs(graph, start):
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in dist:
```

```
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

8.2 Алгоритм Дейкстры

Если у дорог есть длины, BFS уже не подходит. Идея Дейкстры: храним лучшую найденную дистанцию до каждой вершины; всегда берём ближайшую ещё не обработанную вершину и проверяем, не стал ли путь до её соседей короче через неё.



Расстояния от A:

шаг	до A	до B	до C	до D
старт	0	∞	∞	∞
берём A	0	4	1	∞
берём C	0	**3**	1	6
берём B	0	3	1	**4**
берём D	0	3	1	4

Кратчайший путь $A \to D$: $A \to C \to B \to D$ длиной $1 + 2 + 1 = 4$. Путь, который казался коротким ($A \to B \to D$), равен 5.

```
import heapq

def dijkstra(graph, start):
    d = {v: float('inf') for v in graph}
    d[start] = 0
    heap = [(0, start)]                  # расстояние(, вершина)
    while heap:
        du, u = heapq.heappop(heap)
        if du > d[u]:
            continue                      # старая запись, пропускаем
        for v, w in graph[u]:
            if du + w < d[v]:
                d[v] = du + w
                heapq.heappush(heap, (d[v], v))
    return d

g = {'A': [('B', 4), ('C', 1)], 'B': [('A', 4), ('C', 2), ('D', 1)],
     'C': [('A', 1), ('B', 2), ('D', 5)], 'D': [('B', 1), ('C', 5)]}
# dijkstra(g, 'A') -> {'A': 0, 'B': 3, 'C': 1, 'D': 4}
```

Осторожно

Дейкстра работает только если длины рёбер **не отрицательны**.

Где применяется

Навигаторы и карты, маршрутизация данных в интернете, поиск пути в играх, «через сколько рукопожатий» в социальных сетях (BFS).

Задание 11 ●●○

Реализуйте dijkstra и проверьте на графе из рисунка. Затем добавьте ребро A–D длиной 3: как изменился ответ для D?

9. Случайность и хеширование

9.1 Метод Монте-Карло

Иногда проще не считать, а *попробовать* много раз случайно. Бросаем случайные точки в квадрат со стороной 1. Доля точек, попавших в четверть круга радиуса 1, примерно равна $\pi/4$. Значит

$$\pi \approx 4 \cdot \frac{\text{число попаданий}}{\text{число бросков}}.$$

```
import random

def monte_carlo_pi(N):
    inside = 0
    for _ in range(N):
        x, y = random.random(), random.random()
        if x * x + y * y <= 1:
            inside += 1
    return 4 * inside / N
```

Чем больше бросков, тем точнее ответ, но точность растёт медленно: чтобы получить в 10 раз больше точности, нужно в 100 раз больше бросков.

9.2 Хеш-таблицы

Хеш-функция превращает ключ (например, строку) в номер ячейки. Поэтому найти элемент можно почти мгновенно, не перебирая всё подряд. Так устроены dict и set в Python и unordered_map в C++.

Иногда два ключа попадают в одну ячейку — это **коллизия**, и хеш-таблица умеет с ней справляться.

Интересный факт

Парадокс дней рождения: в группе из 23 человек шанс, что у кого-то совпадут дни рождения, больше 50%. Поэтому коллизии в хеш-таблицах — не редкость, а норма.

Где применяется

Моделирование в физике и финансах, компьютерная графика (реалистичный свет), словари и множества, кэши, хранение паролей (хеши).

Задание 12 ●○○

Запустите monte_carlo_pi для $N = 100$, 10^4 и 10^6. Составьте таблицу «N — оценка π». Как меняется точность?

10. Сводка

Алгоритм	Идея	Скорость	Где применяется
Бинарный поиск	делим отсортированный список пополам	$\mathcal{O}(\log n)$	git bisect, базы данных
Пузырёк	меняем неправильных соседей	$\mathcal{O}(n^2)$	для обучения
Слияние	делим, сортируем, сливаем	$\mathcal{O}(n \log n)$	sorted(), большие данные
Жадные	берём лучшее сейчас	обычно быстро	Хаффман, расписания
Динамика	решаем подзадачу один раз и запоминаем	зависит от задачи	diff, опечатки, рюкзак
Евклид	НОД через остатки	$\mathcal{O}(\log n)$	дроби, шифрование
Быстрая степень	возводим в квадрат	$\mathcal{O}(\log n)$	криптография
Решето	вычёркиваем кратные	почти $\mathcal{O}(n)$	простые числа
BFS	обход «волнами»	быстро	кратчайший путь без длин
Дейкстра	ближайшая вершина за вершиной	быстро	навигаторы, сети
Монте-Карло	случайные испытания	медленно сходится	физика, графика
Хеш-таблица	ключ $\to$ ячейка	в среднем мгновенно	dict, set

Главное, что нужно запомнить

1. Алгоритм сравнивают по тому, *как растёт число шагов*, а не по секундам.
2. Хорошая структура данных (отсортированный список, хеш-таблица) делает поиск молниеносным.
3. Жадность быстрая, но её надо проверять на контрпримерах.
4. Если подзадачи повторяются — запоминайте ответы (динамика).

Вопросы для самопроверки

1. Что значит «алгоритм работает за $\mathcal{O}(n^2)$»?
2. Почему бинарный поиск требует отсортированного списка?
3. Чем сортировка слиянием лучше пузырька и почему?
4. Приведите пример, где жадный алгоритм ошибается.
5. Что запоминает динамическое программирование и зачем?
6. Чем BFS отличается от Дейкстры?

11. Ответы к заданиям

№	Ответ
1	(a) n раз, $\mathcal{O}(n)$; (b) $n \cdot n$ раз, $\mathcal{O}(n^2)$; (c) около $\log_2 n$ раз, $\mathcal{O}(\log n)$
2	список из примера: $x = 23 \to$ индекс 5 за 3 шага; $x = 7 \to -1$ (за 3 шага)
3	1000 элементов: 10 шагов; 10^6: 20 шагов
4	при удвоении n время пузырька растёт примерно в 4 раза, слияния — чуть больше чем в 2
5	4 мероприятия: $(1,4), (5,7), (8,11), (12,16)$; правило «самое короткое» ошибается, например, на $(1,5), (4,7), (6,10)$: короткая $(4,7)$ мешает обеим, а лучше взять $(1,5)$ и $(6,10)$
6	способов: $1, 2, 3, 5, 8, \ldots$ ($n = 1$: 1; $n = 2$: 2; $n = 10$: 89; $n = 30$: 1 346 269), это числа Фибоначчи
7	суммы $6, 10, 14, 18$ (нужно $2, 3, 4, 5$ монет, а жадный даёт на одну больше)
8	$252 = 1 \cdot 198 + 54$; $198 = 3 \cdot 54 + 36$; $54 = 1 \cdot 36 + 18$; $36 = 2 \cdot 18$: НОД $= 18$, 4 деления
9	$2^{10} \bmod 1000 = 24$; $3^{13} \bmod 1000 = 323$
10	до 100: 25; до 1000: 168; до 10^6: 78 498
11	$B = 3$, $C = 1$, $D = 4$, путь A, C, B, D; с ребром A–D длиной 3: $D = 3$
12	при $N = 10^6$ получится около 3.14 (отклонение порядка 0.002)

Не заучивайте алгоритмы — понимайте идею, и код напишется сам.